



abdo@soc-lab:~\$ whoami

SOC Home Lab

Detection Engineering and Incident Investigation
on a Self-Built Wazuh and Suricata Lab

Abderrahim ENASSIBI

Cybersecurity and Embedded Systems Engineering
ENSA Tétouan

github.com/bane404/soc-blue-team-lab
linkedin.com/in/abderrahim-enassibi

July 2026

Contents

1	Overview	3
2	Lab Architecture	4
3	SSH Brute Force to Compromise	6
3.1	Timeline	6
3.2	Reconnaissance	6
3.3	Brute Force	6
3.4	Initial Access	7
3.5	Discovery	7
3.6	Evidence	7
3.7	MITRE ATT&CK Mapping	8
3.8	Recommendations	8
3.9	Severity	8
4	File Integrity Monitoring: Inquiry & Fix	9
4.1	Starting Point	9
4.2	Diagnostic Steps	9
4.3	Fix	10
4.4	Verification	10
4.5	What This Actually Was	11
5	SSH Hardening and Vulnerability Detection	12
5.1	SSH Hardening	12
5.2	Vulnerability Detection	13
5.3	Takeaways	14

6	Sigma to Wazuh: Manual Rule Conversion	15
6.1	Confirming There Is No Wazuh Backend	15
6.2	The Rule	15
6.3	Manual Translation to Wazuh XML	16
6.4	Validation	17
6.5	Takeaway	17
7	Custom Detection Rules	18
7.1	Rule 100010 – SSH Brute-Force Escalation	18
7.2	Rule 100020 – Shell Configuration Modification	19
7.3	Summary	19
7.4	Tooling: auditread	19
8	What This Lab Taught Me	21
8.1	Where I’m Focusing Next	21
9	What’s Next	22
9.1	Open Items in the Lab	22
9.2	Continued Practice	22
9.3	Next Project	22

1 Overview

This is a home lab I built to get real hands-on experience with SOC work. It runs on a Debian server with Wazuh as the SIEM and Suricata for network intrusion detection, with a Windows machine connected as an agent. I ran attack simulations against it, wrote detection rules for it, and investigated the alerts it produced.

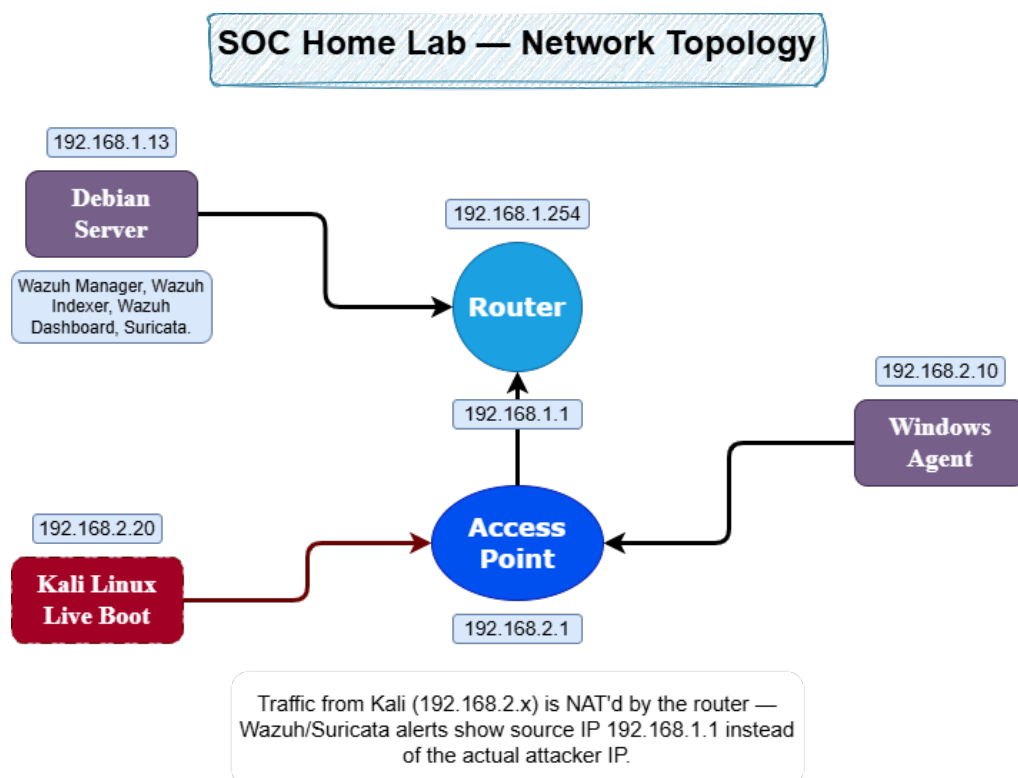
I am a cybersecurity and embedded systems engineering student at ENSA Tétouan. I built this lab because reading about SOC work and doing it are two different things, and I wanted the second one.

Everything in this report ran on real hardware I set up myself: real configuration, real alerts, real bugs I had to debug. Some of it did not work the way it was supposed to the first time, and I wrote that up too instead of only keeping the clean version.

The chapters that follow cover the lab's architecture, then walk through the investigations and detection engineering work in the order that makes the most sense to read, not the order I did them in.

2 Lab Architecture

The lab runs on a Debian 12 server that hosts Wazuh (manager, indexer, and dashboard) and Suricata for network intrusion detection. A Windows machine runs the Wazuh agent and feeds in Windows-side event logs. Attack simulations came from a Kali live-boot USB on the same network, acting as the attacker machine.



One detail worth calling out on the diagram: traffic from the attacker machine gets NAT'd by the router before it reaches the server, so Wazuh and Suricata alerts show the router's IP as the source instead of the attacker's actual IP. It took a while to figure out why the source IP in the alerts did not match what I expected. This is a lab topology detail, not a detection failure.

This has a real consequence for one of the detection rules covered in Chapter 7 – more on that when it comes up.

The hardware, honestly

The Debian server runs on an old Razer laptop with dead fans – I first tried Ubuntu Server on it and hit an NVMe probe bug that Debian 12 didn't have, so Debian became the server OS by necessity, not preference. Cooling is a USB fan pointed at the chassis, and I keep an eye on thermals with `sensors` piped into my `.bashrc` prompt (that's where the `[87.0°C]` in the auditread screenshot in Chapter 7 comes from – not a typo, an actual running temperature). The Kali attacker machine is a live-boot USB I already use as a general toolkit, booted on a separate access point from my main Windows 11 laptop for this lab, not hardware built specifically for it.

3 SSH Brute Force to Compromise

I ran a self-directed attack against my own lab server to see whether the SIEM and IDS stack would actually catch a full attack chain: recon, brute force, login, then post-access enumeration. It caught three of the four stages cleanly. One custom rule I had written failed to fire, which turned out to be a useful finding on its own. The test credential was weak on purpose, for a short window, and reset immediately after.

3.1 Timeline

Time	Event
17:56	First <code>nmap -sV -A</code> scan against the server
18:09	Second <code>nmap</code> scan against the same target
19:19	Hydra SSH brute force started (<code>rockyou.txt</code> , 16 threads)
19:24	Hydra recovers a valid credential
19:28	Successful SSH login with the cracked credential
19:30	Post-login enumeration: <code>whoami</code> , <code>id</code> , <code>uname -a</code> , <code>ps aux</code> , <code>ss -tulnp</code>
19:31	Password reset, exercise closed out

3.2 Reconnaissance

Two `nmap -sV -A` scans from an external Kali host. Suricata flagged the Nmap Scripting Engine user-agent during service probing, and separately flagged probe traffic on port 4444 as trojan-category, because the signature overlaps with known Metasploit handler traffic. Both were ingested and correctly classified by Wazuh.

3.3 Brute Force

Hydra against SSH with the `rockyou.txt` wordlist, 16 parallel threads. The password was recovered in under five minutes. Wazuh's built-in correlation rule 40112 caught the failed-then-successful pattern and raised a single alert covering both the brute force and the valid account login.

Rule 100010 did not fire

A custom rule I had built earlier, 100010, was meant to escalate when Wazuh's base SSH brute-force rule (5712) matched repeatedly inside a 10-minute window. It never fired against this attack. Rule 40112 caught the pattern independently, so nothing was missed operationally – but it's a real tuning gap, worth digging into (full breakdown in Chapter 7).

3.4 Initial Access

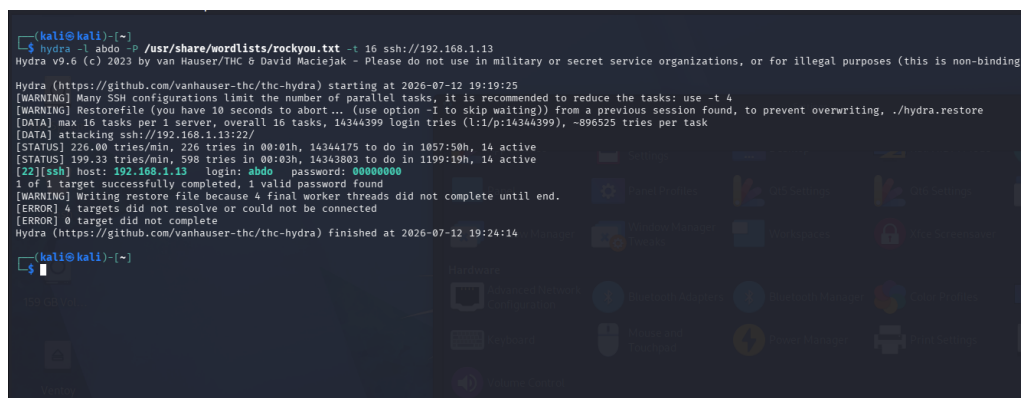
The login itself was logged and attributed correctly by the same rule, 40112.

3.5 Discovery

Post-login enumeration commands did not trigger anything. That is expected: there is no file integrity or audit watch on those paths, and the session was already authenticated by that point. A coverage boundary, not a failure.

3.6 Evidence

- Suricata sig 2024364 – “ET SCAN Possible Nmap User-Agent Observed” (Wazuh rule 86601)
- Suricata sig 3400020 – “POSSBL SCAN SHELL M-SPLOIT TCP”, port 4444 (incidental to nmap probing)
- Wazuh rule 40112 – multiple auth failures followed by success (level 12)
- Hydra output confirming the recovered credential
- Manual enumeration output, captured via screenshot



```
(kali@kali)~$ hydra -l abdo -P /usr/share/wordlists/rockyou.txt -t 16 ssh://192.168.1.13
Hydra v9.6 (c) 2023 by van Hauser/thc & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-12 19:19:25
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://192.168.1.13:22/
[STATUS] 226.00 tries/min, 226 tries in 00:01h, 14344175 to do in 1057:50h, 14 active
[STATUS] 199.33 tries/min, 598 tries in 00:03h, 14343803 to do in 1199:19h, 14 active
[22][ssh] host: 192.168.1.13 login: abdo password: 00000000
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 4 final worker threads did not complete until end.
[ERROR] 4 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-12 19:24:14

(kali@kali)~$
```



```

predecoder.hostname: debian predecoder.program_name: sshd predecoder.timestamp: Jul 12 18:28:13 input.type: log
agent.name: debian agent.id: 000 manager.name: debian data.srcip: 192.168.1.1 data.dstuser: abdo data.srcport: 47506
rule.mail: true rule.level: 12 rule.pci_dss: 10.2.4, 10.2.5, 11.4 rule.hipaa: 164.312.b rule.tsc: CC6.1, CC6.8, CC7.2, CC
7.3 rule.description: Multiple authentication failures followed by a success. rule.groups: syslog, attacks
rule.nist_800_53: AU.14, AC.7, SI.4 rule.frequency: 2 rule.gdpr: IV_35.7.d, IV_32.2 rule.firedtimes: 3

```

3.7 MITRE ATT&CK Mapping

Tactic	Technique	Evidence
Reconnaissance	T1046 – Network Service Discovery	Suricata 2024364, 3400020 → Wazuh 86601
Credential Access	T1110.001 – Brute Force: Password Guessing	Wazuh 40112
Initial Access	T1078 – Valid Accounts	Wazuh 40112
Discovery	T1082 – System Information Discovery	Manual observation, not alert-generating under current config

3.8 Recommendations

1. Move SSH to key-based auth and disable password login – attempted separately, reverted for now (covered in the hardening chapter).
2. Re-tune 100010's frequency and timeframe thresholds for fast, parallel-threaded attacks, or retire it in favor of relying on 40112.
3. Consider audit watches or command monitoring on sensitive post-login commands (`ps`, `ss`, `netstat`), if that visibility is worth the added noise.
4. Rate-limit or fail2ban SSH to shrink the practical window for automated credential attacks, regardless of password strength.

3.9 Severity

High as simulated. A real-world equivalent – valid SSH access, especially with sudo membership – would be a critical initial-access foothold. The actual risk here was nil: the weak credential was intentional, time-boxed, and reset immediately after.

4 File Integrity Monitoring: Inquiry & Fix

FIM alerts were not firing reliably even though the `syscheck` config was correct. I traced it to `wazuh-analysisd` silently dropping events because of a JSON decoder field limit, triggered by Suricata's verbose `eve.json` output overloading the manager. Raising the decoder limit fixed it: both general FIM (rule 550) and a custom auditd-based rule (100020) now fire correctly and reliably.

4.1 Starting Point

`syscheck` was configured correctly from early on: realtime watches on `/etc`, `/usr/bin`, and `/home`, with `scan_on_start` enabled. But test file changes were not producing alerts, and there was no obvious config error, which made this a genuinely confusing one to debug.

4.2 Diagnostic Steps

1. Confirmed `syscheck` itself was running and had a real baseline

```
sudo sqlite3 /var/ossec/queue/fim/db/fim.db "SELECT COUNT(*) FROM file_entry;"
```

Returned 8,586 entries. `syscheck` was scanning and tracking files correctly, so the problem was not on the scanning side.

2. Turned on debug logging and watched a live scan

With `syscheck.debug=2` set and the manager restarted, the log showed `syscheck` actively sending integrity-check messages for its baseline scan, working as expected. But interleaved with those messages was a wall of:

```
wazuh-analysisd: ERROR: Too many fields for JSON decoder.
```

`wazuh-analysisd` is the component that turns incoming events into alerts. It was rejecting messages that exceeded its default field limit (256 fields per JSON decoder). This was not specific to `syscheck`'s messages – anything with enough fields was getting dropped.

3. Traced the actual source

This is a known issue tied to Suricata's `eve.json` integration. Suricata logs verbose per-flow metadata (HTTP, TLS, flow stats) that can exceed the decoder's field ceiling. Since `eve.json` ingestion had been added into `ossec.conf` earlier in the lab, that was almost certainly what was periodically overloading `analysisd` and disrupting its ability to keep up with other event processing, FIM alerts included.

4.3 Fix

Raised the decoder field limit in `/var/ossec/etc/local_internal_options.conf`:

```
analysisd.decoder_order_size=1024
```

Restarted the manager. This does not always fully eliminate the error under sustained load – the real long-term fix is trimming Suricata's `eve.json` fields at the source – but it was enough to clear the backlog and restore alerting for a normal workload.

4.4 Verification

General FIM – rule 550

Confirmed via the dashboard: alert history for rule 550 goes back to before this fix, meaning baseline FIM detection was already working intermittently. Two additional test changes to `/etc/hosts` produced clean, complete alerts with full before/after checksums (md5, sha1, sha256) and correct MITRE mapping (T1565.001).

Rule 100020 confirmed end to end

This was the actual unresolved gap: the rule had never fired, despite the auditd watch (`auditctl -w /home/abdo/.bashrc -p wa -k shell_config_mod`) being correctly registered. I traced the full chain manually:

1. `sudo ausearch -k shell_config_mod` – confirmed auditd logged the syscall against `.bashrc` with the correct key
2. Checked Wazuh's ingestion of `/var/log/audit/audit.log` via the auditd decoder
3. Checked `alerts.json` directly for a match

After a real trigger (appending a line to `.bashrc`), the alert fired correctly:

```
"rule":{"level":7,"description":"Shell configuration file modified: /home/abdo/.bashrc","id":"100020",
"mitre":{"id":["T1546.004"],"tactic":["Privilege Escalation","Persistence"]}}
```

Full chain confirmed: audit watch → auditd log entry → Wazuh's auditd decoder → custom rule match → alert with correct MITRE mapping.

4.5 What This Actually Was

Not a syscheck configuration problem. The root cause was `analysisd` getting overloaded by oversized JSON from the Suricata integration, which appears to have delayed or dropped processing across multiple event sources, not just FIM. The fix – raising the decoder limit – addressed the symptom well enough to restore reliable alerting. The proper long-term fix is reducing the size of what Suricata sends to `eve.json` in the first place, which has not been done yet.

Open item

Suricata's `eve.json` output is still verbose enough to occasionally hit the decoder ceiling under bursty traffic, confirmed by a large error count during one test session. Trimming the `eve-log` field set in `/etc/suricata/suricata.yaml` down to essentials (alert metadata, minimal flow/HTTP/TLS fields) is the more durable fix and has not been implemented yet.

5 SSH Hardening and Vulnerability Detection

This chapter covers two related pieces of work: hardening SSH on the lab server after the brute-force exercise, and standing up Wazuh’s vulnerability detection module, which surfaced a real critical CVE on the Windows agent that got patched as part of this.

5.1 SSH Hardening

What was applied

- `PermitRootLogin no`
- `MaxAuthTries 3`
- Password authentication left enabled, kept on port 22

Key-based auth: attempted, reverted

I tried moving to key-only authentication across three devices: the lab server, a WSL client, and a phone terminal. I ran into a reproducible problem – manually copying public keys across devices, especially typing or pasting from a mobile terminal, introduced silent corruption in `authorized_keys` (a stray space mid-key, and at one point a private key pasted into the wrong file). `ssh-keygen -lf` on the server would silently skip malformed entries rather than error, which made the failure hard to diagnose from the client side. Only `ssh -vvv` plus checking fingerprints directly against `authorized_keys` made the actual mismatch visible.

After getting it working for two devices and hitting the same corruption pattern a third time, I reverted to password authentication with `MaxAuthTries` in place. For a single-operator lab on a private LAN, managing keys across three different devices was not worth the hassle yet. I deleted the keys everywhere and went back to password auth. I will probably set up keys again once there is an actual reason to – multiple users, or the server ever being reachable from outside the LAN.

5.2 Vulnerability Detection

Config correction

The setup guide I was originally following referenced Wazuh's old `<vulnerability-detector>` block, with per-OS `<provider>` tags. That syntax was deprecated in Wazuh 4.8.0. On 4.14.6, the correct block is:

```
<vulnerability-detection>
  <enabled>yes</enabled>
  <index-status>yes</index-status>
  <feed-update-interval>60m</feed-update-interval>
</vulnerability-detection>
```

No OS-specific provider tags needed.

Enabling and verifying

The indexer connection was already set up and reachable, so I restarted the manager and checked the log to confirm the module actually started, instead of assuming it worked:

```
wazuh-modulesd:vulnerability-scanner: INFO: Vulnerability scanner module started.
wazuh-modulesd:vulnerability-scanner: INFO: Initiating update feed process.
wazuh-modulesd:vulnerability-scanner: INFO: Triggered a re-scan after content update.
wazuh-modulesd:vulnerability-scanner: INFO: Feed update process completed.
```

A bit later, the dashboard had populated: 101 findings on the abdo101 agent, matched against real installed software (Chrome, Python, Wireshark, ASP.NET Core, Steam).

Severity	Count
Critical	2
High	58
Medium	34
Low	3
Total	101

CVE-2025-55315 – CVSS 9.9

One of the two critical findings, and Microsoft’s highest-ever CVSS score for an ASP.NET Core vulnerability. It is an HTTP request smuggling bug in Kestrel/ASP.NET Core caused by inconsistent parsing of chunked transfer-encoding, letting a crafted request hide a second request inside it and potentially bypass authentication or authorization checks in whatever application sits on top. The severity score reflects worst-case impact for internet-facing apps behind a reverse proxy – on a personal workstation running no exposed services, actual exploitability is close to nil, but it was a one-command fix, so worth doing regardless.

The agent had two affected runtime versions installed:

- ASP.NET Core 8.0.0 – vulnerable, fix available (8.0.21+)
- ASP.NET Core 6.0.8 – vulnerable, but .NET 6 is end-of-life; no patch exists or will be released

Action taken: updated the 8.0 runtime to 8.0.28 via the official Microsoft installer. Confirmed via `dotnet -list-runtimes` post-install, and cross-checked that the CVE cleared from the next Wazuh inventory cycle.

Action on the 6.0.8 finding: no dependency currently relies on .NET 6 on this machine. Documented as an accepted risk rather than remediated – there is no vendor fix to apply, and the correct response to an EOL, unpatched component is either removal or a documented risk acceptance, not waiting indefinitely for a patch that is not coming. Flagged for removal if anything surfaces as depending on it.

5.3 Takeaways

- Vendor documentation ages quickly for actively developed tools like Wazuh. Checking current docs against config before applying anything saved a wasted step here.
- Key-based SSH auth across heterogeneous devices (desktop, WSL, mobile) has real friction that is worth weighing against the actual threat model before committing to it, especially for a single-operator lab.
- Vulnerability scanning is only useful if findings get triaged, not just displayed. The difference between a clean-looking dashboard and having actually looked at what is on it matters more than the finding count.

6 Sigma to Wazuh: Manual Rule Conversion

The plan was to pull a Sigma rule from the official SigmaHQ repository and convert it to Wazuh XML using `sigma-cli`. That tool does not actually support Wazuh as a conversion target – confirmed directly rather than assumed – so the rule was translated by hand instead. This ended up being a better exercise for understanding rule logic than a straight automated conversion would have been.

6.1 Confirming There Is No Wazuh Backend

```
$ sigma plugin list
```

No Wazuh backend in sigma-cli

The full plugin list includes backends for QRadar, Cortex XDR, Carbon Black, SentinelOne, Splunk, Elasticsearch, OpenSearch, CrowdStrike, Kusto, Datadog, Panther, Logpoint, SQLite, and about a dozen more. No `wazuh` entry anywhere in the list.

Wazuh's indexer is built on OpenSearch, and `sigma-cli` does support an OpenSearch backend, but that produces OpenSearch alerting-rule syntax, not Wazuh's own `local_rules.xml` format. The two are not interchangeable – Wazuh detection rules and OpenSearch dashboards or alerts are different systems under the same platform. Even the closest available backend does not actually solve the problem.

6.2 The Rule

Source: `lnx_auditd_unix_shell_configuration_modification.yml` from the SigmaHQ repository. It detects modification of shell configuration files, a common persistence technique where an attacker who has gained access appends commands that execute every time a new shell is opened.

```
title: Unix Shell Configuration Modification
id: a94cdd87-6c54-4678-a6cc-2814ffe5a13d
status: test
description: Detect unix shell configuration modification. Adversaries may
  establish persistence through executing malicious commands triggered when
  a new shell is opened.
tags:
  - attack.privilege-escalation
```



```

- attack.persistence
- attack.t1546.004
logsource:
  product: linux
  service: auditd
detection:
  selection:
    type: 'PATH'
    name:
      - '/etc/shells'
      - '/etc/profile'
      - '/etc/bash.bashrc'
      - '/etc/zsh/zshrc'
      - '/root/.bashrc'
      - '/home/*/.*bashrc'
      - '/home/*/.*zshrc'
      (and about a dozen more shell config paths)
    condition: selection
falsepositives:
  - Admin or User activity are expected to generate some false positives
level: medium

```

The Sigma rule watches roughly two dozen shell config paths across bash, zsh, and csh, system-wide and per-user. For the lab, this was narrowed to a single path relevant to the actual environment (`/home/abdo/.bashrc`) rather than replicating the full path list – a deliberate scoping decision for a single-user lab, not a limitation of the translation itself.

6.3 Manual Translation to Wazuh XML

`local_rules.xml`, rule 100020:

```

<group name="local,audit,">
  <rule id="100020" level="7">
    <if_sid>80700</if_sid>
    <field name="audit.key">shell_config_mod</field>
    <description>Shell configuration file modified: $(audit.file.name)</description>
    <mitre>
      <id>T1546.004</id>
    </mitre>
    <group>persistence,local_rules,</group>
  </rule>
</group>

```

The Sigma rule matches on an explicit list of file paths, filtering right there in the detection block. Wazuh's approach is different: rather than pattern-matching a path at rule-evaluation time, the audit watch itself is scoped to the target file with a specific key, set at the auditd level:

```
auditctl -w /home/abdo/.bashrc -p wa -k shell_config_mod
```

Wazuh splits what Sigma does in one place into two: the path-watching happens earlier, in the auditd config itself, and the Wazuh rule just checks for that key showing up in the logs. So

instead of one rule doing everything, the “what to watch” part and the “alert on it” part live in two different places. That is really what the manual conversion came down to – not translating line by line, but figuring out which platform expects which part of the logic.

6.4 Validation

Triggered by appending to the watched file and confirming the alert fired with the correct rule ID, MITRE mapping, and file path in the alert body. Rule fires as expected.

6.5 Takeaway

Automated conversion tools are only as good as their backend coverage. It is worth checking `sigma plugin list`, or the equivalent for whatever tool is in play, before assuming a conversion path exists, rather than debugging a command that was never going to work. Doing the translation by hand also forced a clearer understanding of how Wazuh’s rule-matching model (base rule plus field or key matching) differs structurally from Sigma’s declarative field-matching model, something a working `sigma convert` command would have hidden.

7 Custom Detection Rules

Two custom rules live in `custom-rules/local_rules.xml`: one escalates on repeated SSH brute-force detections from the same source, the other flags shell configuration file changes as a persistence indicator.

```
<group name="local,syslog,sshd,">
  <rule id="100010" level="12" frequency="2" timeframe="600">
    <if_matched_sid>5712</if_matched_sid>
    <same_source_ip/>
    <description>Repeated SSH brute force detections from same source -- escalate to
      Tier 2</description>
    <mitre>
      <id>T1110.001</id>
    </mitre>
    <group>authentication_failures,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>

<group name="local,audit,">
  <rule id="100020" level="7">
    <if_sid>80700</if_sid>
    <field name="audit.key">shell_config_mod</field>
    <description>Shell configuration file modified: $(audit.file.name)</description>
    <mitre>
      <id>T1546.004</id>
    </mitre>
    <group>persistence,local_rules,</group>
  </rule>
</group>
```

7.1 Rule 100010 – SSH Brute-Force Escalation

This rule watches for Wazuh's own base SSH brute-force rule, 5712, matching twice from the same source IP inside a 10-minute window (`frequency="2"`, `timeframe="600"`), and escalates that pattern to level 12. It is meant as a second layer on top of Wazuh's built-in correlation logic: a way to flag repeated brute-force attempts from one source crossing a threshold, tagged against T1110.001 and the relevant PCI DSS groups.

It has a known tuning gap. Against a fast, parallel-threaded Hydra attack, it never fired (see Chapter 3) – the 2-matches-in-600-seconds window assumes a slower, sequential brute-force pattern, not a fast automated one. Wazuh's own correlation rule (40112) caught the attack independently, so nothing was missed operationally, but 100010 itself has not been re-tuned

yet.

There is a second problem with this rule that is specific to this lab's topology, not to the rule logic itself: the router NAT's all attacker traffic before it reaches the server (Chapter 2), so every session from the Kali box shows up as the same source IP regardless of which real machine sent it. That makes `same_source_ip` trivially true in this environment – it is not actually discriminating between attackers on a NAT'd LAN. Off this LAN, on a network where attacker source IPs are genuinely distinct, the check would do real work. Here, it is currently redundant with the frequency/timeframe logic alone.

7.2 Rule 100020 – Shell Configuration Modification

This rule fires on Wazuh's base auditd rule, 80700, filtered down to events where `audit.key` equals `shell_config_mod`. That key is set entirely outside of Wazuh, by the auditd watch itself:

```
auditctl -w /home/abdo/.bashrc -p wa -k shell_config_mod
```

The rule does not need to know the file path in advance – it just checks for the key, and pulls the actual path into the alert description via `$(audit.file.name)`. Mapped to T1546.004, persistence through shell configuration.

Confirmed firing end to end (see Chapter 4): audit watch → auditd log entry → Wazuh's auditd decoder → rule match → alert with the correct MITRE mapping.

7.3 Summary

Rule	Trigger	MITRE	Status
100010	Rule 5712 matched twice from the same source within 10 minutes	T1110.001	Needs tuning
100020	<code>audit.key = shell_config_mod</code>	T1546.004	Confirmed working

7.4 Tooling: auditread

Raw `ausearch` output is dense and hard to scan quickly during an investigation – `SYSCALL`, `PATH`, `CWD`, and `PROCTITLE` records print as separate lines that have to be mentally stitched back together per event. I built a small wrapper, `auditread`, that groups each event into one readable block (when, what, who, where, how, and the actual command) so I could scan audit activity faster while working in the lab.

```
auditread [-k key] [-n N] [-x] [-s yes|no] [-u user]
```

Flag	Meaning
-k key	filter by audit key (e.g. <code>shell_config_mod</code>)
-n N	show N events (newest first by default)
-x	show oldest first instead of newest
-s yes no	filter by success/failure
-u user	filter by audit username

Example:

```
auditread -k shell_config_mod -n 3
```

```
abdo@debian [87.0°C] ~$ sudo auditread -k shell_config_mod -n 3
```

```
auditread - showing 3 of 13 event(s)
filter: -k shell_config_mod
order: newest first

[WHEN] 07/16/2026 00:20:58
[WHAT] config-change | add_rule | success=yes
[WHO] uid=root | audit=unset
[WHERE] auditd config
[HOW] auditctl @ /usr/sbin/auditctl
[CMD] /sbin/auditctl -R /etc/audit/audit.rules
[CWD] /

-----
[WHEN] 07/15/2026 13:31:45
[WHAT] config-change | add_rule | success=yes
[WHO] uid=root | audit=unset
[WHERE] auditd config
[HOW] auditctl @ /usr/sbin/auditctl
[CMD] /sbin/auditctl -R /etc/audit/audit.rules
[CWD] /

-----
[WHEN] 07/14/2026 20:30:33
[WHAT] syscall | openat | success=yes
[WHO] uid=abdo | audit=abdo
[WHERE] /home/abdo/.bashrc
[HOW] bash @ /usr/bin/bash
[CMD] -bash
[CWD] /home/abdo
```

Built with AI assistance – disclosed

I drafted the parsing logic with AI assistance rather than writing the `ausearch` output parsing from scratch: the field-extraction regex and event-grouping logic were AI-generated based on a description of what I wanted the tool to do. I tested it against real audit output from the lab, adjusted the filters and output format to match what was actually useful during investigations, and use it regularly as part of the lab workflow.

8 What This Lab Taught Me

I built this lab to actually do SOC work, not just read about it. Here is what stuck.

- **Stood up a real SIEM stack end to end** – Wazuh manager, indexer, and dashboard, plus Suricata for network detection, with agents deployed on both Linux and Windows (Chapter 2).
- **Ran a full attack chain against my own infrastructure and read the results.** – recon through compromise, then went through what the stack caught, what it missed, and why (Chapter 3).
- **Debugged a real infrastructure failure by tracing it.** – FIM alerts were silently failing for a non-obvious reason (an overloaded JSON decoder), and I ran it down through logs instead of re-checking the same config repeatedly (Chapter 4).
- **Practiced applied vulnerability management** – found and patched a critical real-world CVE on a live agent, and made a documented risk-acceptance call on an unpatchable EOL component instead of leaving it unaddressed (Chapter 5).
- **Learned Wazuh’s detection model well enough to translate rules by hand** – confirmed a real gap in the tooling (no working Sigma-to-Wazuh backend) rather than assuming it existed, then did the conversion manually (Chapter 6).
- **Wrote and validated custom detection rules, and found a real gap in my own work** – one of my two rules didn’t fire against a fast attack, and I traced exactly why instead of letting it slide (Chapter 7).

8.1 Where I’m Focusing Next

- **Zeek** – Suricata is running well, but I want the habit of generating and querying `conn.log`, `dns.log`, and `http.log` alongside it.
- **DFIR** – basic disk forensics with Autopsy and memory analysis with Redline, both still untouched. I also want to review the NIST IR phases properly rather than half-remembering them.
- **Windows depth** – I’m solid on Event IDs 4624 and 4625; the rest of the set, and what Sysmon adds over native logging, is next.
- **OpenSearch dashboards** – I’ve only used Wazuh’s default views so far. Building a custom one is on the list.

9 What's Next

Two things are still open in the lab itself. After that, it's closing the gaps above and building the next thing.

9.1 Open Items in the Lab

Item	Context	Next step
<code>eve.json</code> trimming	Suricata's verbose output overloaded <code>analysisd</code> 's decoder under load (Chapter 4)	Trim the <code>eve-log</code> field set in <code>suricata.yaml</code> down to essentials
SSH key-based auth	Reverted after repeated <code>authorized_keys</code> corruption across three devices (Chapter 5)	Revisit with a cleaner key-transfer method instead of manual copy-paste
Rule 100010 tuning	Known gap, full detail in Chapter 7	Re-tune the frequency/timeframe thresholds, or retire it in favor of rule 40112

9.2 Continued Practice

I am continuing with BTLO and CyberDefenders challenges alongside this. Those writeups live in a separate repository, `security-writeups`, rather than this one.

9.3 Next Project

Next up is a honeypot – attract real traffic, log what it actually does, and see what a live internet-facing target looks like compared to attacks I've been running myself. Open to a different direction if something better comes up first, but that's the plan.